CO523 – Programming Languages

Mini Project Report

Design and Implementation of a C-Lite Interpreter

E/20/254
Methpura S.K.P.

Abstract

This project presents the design and implementation of an interpreter for C-Lite, a simplified subset of the C programming language. The goal of the project is to demonstrate the core phases of language processing, including lexical analysis, syntax analysis, and semantic evaluation.

The implementation is written in Python and organized into three major components: a lexer, a parser, and an interpreter. The lexer scans the input program and converts it into tokens. The parser validates the token stream using a recursive-descent parsing approach and constructs an Abstract Syntax Tree (AST). The interpreter then executes the AST while maintaining a symbol table that stores variable bindings.

The implemented language supports variable declarations, assignments, arithmetic expressions, relational and logical operators, conditional control flow using if-else statements, block statements, boolean literals, and a built-in printf statement for output.

This project provides practical insight into how programming languages transform source text into executable behavior.

1. Introduction

Programming language implementation typically involves several stages that transform high-level source code into executable behavior. These stages include lexical analysis, syntax analysis, and semantic evaluation.

This project focuses on building a small interpreter for C-Lite, a simplified subset of the C programming language. Unlike compilers that translate programs into machine code, interpreters execute programs directly by evaluating their structure.

The interpreter developed in this project follows a traditional language processing pipeline:

- Lexical Analysis – the source code is scanned and converted into tokens.
- Syntax Analysis – the tokens are validated using a formal grammar and an AST is constructed.
- Semantic Evaluation – the AST is executed and program behavior is produced.

The implementation demonstrates how fundamental language constructs such as variables, expressions, control flow, and output can be processed by a language interpreter.

2. Objectives

The objectives of this project are:

- To demonstrate practical understanding of the translation process from source code to execution.
- To implement a Lexical Analyzer (Scanner) that converts raw program text into tokens.
- To implement a Parser that validates token sequences against a grammar specification.
- To construct an Abstract Syntax Tree (AST) representing program structure.
- To implement semantic evaluation using a symbol table.
- To support essential programming constructs such as declarations, assignments, expressions, and conditional statements.
- To analyze implementation decisions such as operator precedence handling and scope management.

3. Overview of the Implemented Language

The implemented C-Lite subset supports the following features:

Data Types

- int
- float
- boolean literals true and false

Statements

- variable declaration
- assignment
- conditional statements (if, else, else if)
- block statements { }
- output using printf
- Operators

Arithmetic operators: + - * /

Relational operators: < > <= >=

Equality operators: == !=

Logical operators: && || !

Example Program

```
int x;
x = 10;
if (x > 5) {
    x = x - 1;
} else if (x == 5) {
```

```
    x = x + 2;
} else {
    x = x + 3;
}
printf(x);
```

4. System Architecture

The interpreter is divided into three main modules.

4.1 Lexical Analysis (Scanner)

The lexical analyzer is implemented in *lexer.py.*

The lexer reads the source code character by character and groups characters into tokens.

The lexer recognizes:

- Keywords: if, else, int, float, true, false, printf
- Identifiers
- Numeric literals - integers and floating-point numbers
- Operators
- Symbols ( ) { } ; ,

Multi-character Operators

The lexer also supports operators such as:

== != <= >= && ||

This is implemented using a peek mechanism, which checks the next character before creating a token.

Each token is represented using the Token class defined in: *clite_token.py*

Example token representation:

Token(KEYWORD, int)

Token(IDENTIFIER, x)

Token(OPERATOR, ==)

Token(NUMBER, 10)

4.2 Syntax Analysis (Parser)

The syntax analyzer is implemented in parser.py using a recursive-descent parser.

The parser validates the token sequence according to the grammar and constructs an Abstract Syntax Tree (AST).

AST Node Types

The following node types are defined:

- Program
- Declaration
- Assignment
- IfStatement
- Block
- Expression
- Identifier
- Number
- Boolean
- Printf

The entry point of the parser is: parse_program()

which parses: declarations → statements

4.3 Semantic Evaluation (Interpreter)

The semantic evaluation phase is implemented in interpreter.py.

The interpreter traverses the AST and performs program execution.

Symbol Table

A symbol table is implemented using a Python dictionary: self.symbol_table = {}

It stores variable names and their corresponding values.

Example: { 'x': 9 }

Execution Rules

The interpreter performs:

- variable declaration
- assignment evaluation
- expression evaluation
- conditional branching
- output generation

Output is handled by the printf node: visit_printf()

Values are printed to the console and also stored in an output list for testing.

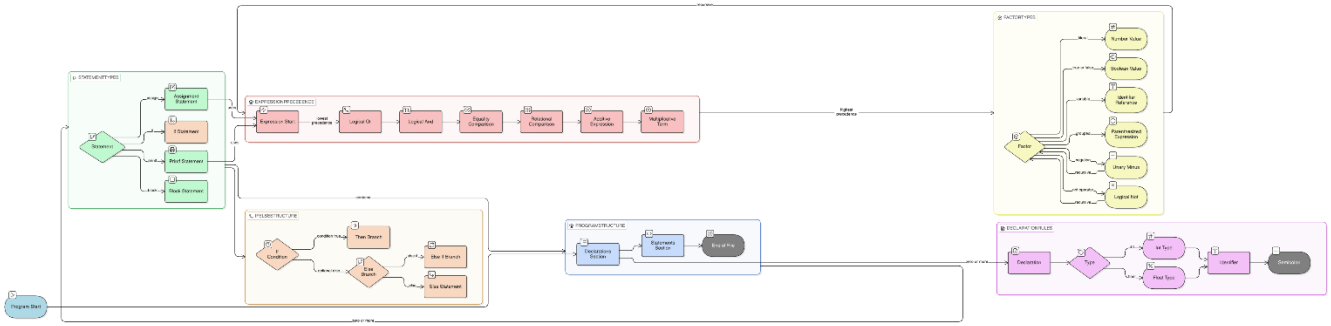
5. Grammar Specification (EBNF)

The implemented grammar of C-Lite is defined using Extended Backus-Naur Form (EBNF).

program = declarations, statements, EOF ;

declarations = { declaration } ;

```
declaration     = type, identifier, ";" ;

type   = "int" | "float" ;

statements   = { statement } ;

statement = assignment

            | if_statement

            | printf_statement

            | block ;

assignment      = identifier, "=", expression, ";" ;

printf_statement = "printf", "(", expression, ")", ";" ;

if_statement    = "if", "(", expression, ")", statement,

              [ "else", ( if_statement | statement ) ] ;

block          = "{", statements, "}" ;

expression  = logical_or ;

logical_or  = logical_and, { "||", logical_and } ;

logical_and     = equality, { "&&", equality } ;

equality  = relational, { ( "==" | "!=" ), relational } ;

relational  = additive, { ( "<" | ">" | "<=" | ">=" ), additive } ;

additive= term, { ( "+" | "-" ), term } ;

term  = factor, { ( "*" | "/" ), factor } ;

factor = number

            | boolean

            | identifier

            | "(", expression, ")"

            | "-", factor

            | "!", factor ;

boolean = "true" | "false" ;

identifier= letter, { letter | digit | "_" } ;

number = integer | float ;
```



6. Design Discussion

6.1 Choice of Recursive-Descent Parsing

Recursive-descent parsing was selected because it is easy to implement and directly maps grammar rules to parser functions.

Each grammar rule corresponds to a parsing function such as:

parse_expression()

parse_statement()

parse_if_statement()

This makes the parser easier to debug and extend.

6.2 Operator Precedence Handling

Handling operator precedence is a key challenge in expression parsing.

The solution used in this project is precedence-based parsing, where expression parsing is divided into multiple levels:

- logical OR
- logical AND
- equality
- relational
- additive
- multiplicative
- factors

This ensures expressions are evaluated correctly.

Example: 3 + 4 * 5

is interpreted as: 3 + (4 * 5)

6.3 Unary Operator Handling

Unary operators are supported: - !

Unary minus is implemented as: 0 - x

Logical NOT is implemented by evaluating the boolean negation of the operand.

6.4 Else-If Chains

Else-if statements are implemented recursively.

Example:

```
if (...) {...}
else if (...) {...}
else {...}
```

is internally represented as nested IfStatement nodes.

This naturally resolves the dangling-else problem.

6.5 Symbol Table Design

The interpreter uses a single global symbol table.

Advantages:

- simple implementation
- easy variable lookup

Limitation:

- no block-level scope
- variables inside blocks remain globally visible

Future versions could implement nested symbol tables.

7. Test Suite

The interpreter was verified using several sample programs.

Test 1: Arithmetic Evaluation

```
source_code = """
int x;
x = 2 + 3 * 4;
printf(x);
"""
```

```
PS D:\Semester7\CO523- Programming Languages\miniProject> python main.py
14
```

This verifies operator precedence.

Test 2: Conditional Execution

```
source_code = """
int x;
x = 10;

if (x > 5) {
    x = x - 1;
}

printf(x);      You, 6 days ago • finaloze …
"""
```

```
PS D:\Semester7\CO523- Programming Languages\miniProject> python main.py
9
```

This verifies conditional execution.

Test 3: Else-If Chain

```
source_code = """
int x;
x = 5;

if (x > 5) {
    printf(1);
}
else if (x == 5) {
    printf(2);
}
else {
    printf(3);
}       You, last week • lexer …
"""
```

```
PS D:\Semester7\CO523- Programming Languages\miniProject> python main.py
2
```

This verifies else-if parsing and evaluation.

Test 4: Logical Expressions

```
source_code = """
int x;
x = 10;

if (x > 5 && x < 20) {
    printf(1);
}
"""
```

```
PS D:\Semester7\CO523- Programming Languages\miniProject> python main.py
1
```

This verifies logical operators.